arXiv:2406.14197v2 [cs.CL] 24 Jan 2025

On the Representational Capacity of Neural Language Models with Chain-of-Thought Reasoning

Franz Nowak* **Anej Svete*** **Alexandra Butoi** **Ryan Cotterell**
{franz.nowak, anej.svete, alexandra.butoi, ryan.cotterell}@inf.ethz.ch
ETH zürich

Abstract

The performance of modern language models (LMs) has been improved by chain-of-thought (CoT) reasoning, i.e., the process of generating intermediate results that guide the model towards a final answer. A possible explanation for this improvement is that CoT reasoning extends an LM's computational power, as RNNs and transformers with additional scratch space are known to be Turing complete. Comparing LMs to Turing machines, however, introduces a category error—Turing machines decide language membership, whereas LMs define *distributions* over strings. To bridge this gap, we formalize CoT reasoning in a probabilistic setting. We present several results on the representational capacity of recurrent and transformer LMs with CoT reasoning, showing that they can represent the same family of distributions over strings as *probabilistic* Turing machines.

https://github.com/rycolab/cot-lms

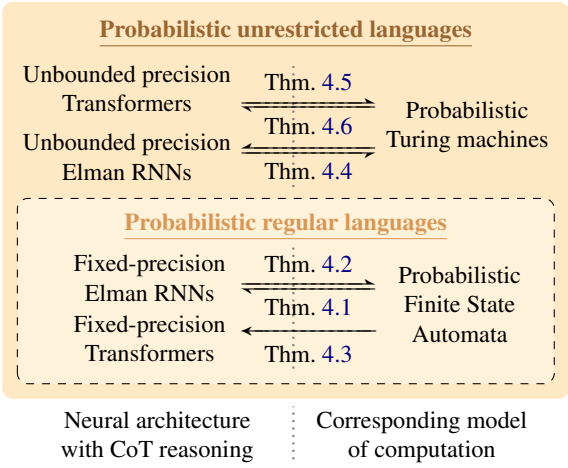

Figure 1: A schematic overview of our main results, showing which neural language models extended with CoT reasoning (left) correspond to which probabilistic models of computation (right).

1 Introduction

Motivated by how humans solve complex problems by noting down intermediary results, Wei et al. (2022b) introduced **chain-of-thought** (CoT) reasoning.[1] CoT reasoning helps language models (LMs) solve reasoning tasks by allowing them to store intermediary results in a scratch space that is *not* part of the final output. The empirical success of CoT reasoning has made it an established component of modern neural LMs (Nye et al., 2021; Wei et al., 2022a,b; Suzgun et al., 2022; Kojima et al., 2023; Wies et al., 2023)—seemingly overnight. Such empirical success motivates a thorough understanding of the abilities of CoT-augmented LMs. While existing theoretical treatments have shed light on some aspects of this framework, we are still far from a concrete theoretical understanding of CoT (Feng et al., 2023).

CoT reasoning can be interpreted as encouraging LMs to perform additional sequential computation steps while storing the result of that computation. Naturally, this suggests that CoT reasoning should be formalized in a manner that exploits our understanding of well-known, stateful models of computation. Theory of computation provides a natural toolbox for this avenue of exploration. Indeed, the internal configurations of many neural language models have been previously linked to intermediate steps in neural LMs (Pérez et al., 2021; Merrill and Sabharwal, 2024; Feng et al., 2023).

A **language model** is definitionally a distribution over strings from some fixed alphabet.[2] This definition is somewhat at odds with the way existing work has investigated the representational capacity of CoT-augmented LMs. Firstly, most results in this area concern unweighted language *recognition*, which is different from the inherently probabilistic task of language modeling. Secondly, Turing completeness results such as Pérez et al.'s (2021) construction—a seminal result showing the Turing completeness of transformer LMs—usually require *additional symbols* not present in the original alphabet to correctly simulate a Turing machine. Because the output alphabets of the transformer LM and the Turing machine it simulates fundamentally differ, it is difficult

*Equal contribution.

[1]We use the more general term CoT *reasoning* over the original term CoT *prompting* as prompting is just one way to elicit CoT reasoning (Wang and Zhou, 2024).

[2]An alphabet is a finite, non-empty set.

to discuss their *equivalence*. This motivates a more fine-grained treatment of the representational capacity of neural LMs in which: *(1)* LMs are treated as *probabilistic* models of computation, i.e., they assign weights to strings rather than deciding language membership, and *(2)* language model equivalence is defined on the output string level, and the analysis takes into account the additional information required to be encoded in additional outputs to achieve the equivalence.

We address the first point by analyzing CoT-augmented LMs using *probabilistic* models of computation, which provide a convenient way of describing families of distributions over strings with standard models of sequential reasoning. To address the second point, we define a new type of relationship between language models, which we call **regular reducibility**. Intuitively, an LM is regularly reducible into another one if the strings generated by the latter are sufficiently simple transformations of those generated by the former. An instance of such a transformation is the deletion of the intermediary computation steps of CoT reasoning. Formalizing CoT reasoning in this framework, we find that it increases the computational power in both RNN and transformer LMs. Concretely, we find that CoT-augmented constant-precision RNN LMs are equivalent to probabilistic finite-state automata (PFSAs). This is in contrast to the constant-precision RNN LMs *without* CoT reasoning, which are equivalent to *deterministic* PFSAs (Svete and Cotterell, 2023). Additionally, we show how Turing-complete linear-precision RNN LMs can be thought of as performing CoT reasoning. Finally, we show that both linearly bounded precision RNN LMs and logarithmically bounded precision decoder-only transformer LMs with CoT-reasoning can simulate any probabilistic Turing machine. Taken together, our results frame CoT reasoning in pure language modeling terms and describe its intuitive and formal connections to probabilistic models of computation.

2 Preliminaries

In our analysis, we assume rational (rather than real) arithmetic for all definitions and computations.

2.1 Language Models

A **formal language** L is a subset of the Kleene closure Σ^* of some alphabet Σ. We call an element $\boldsymbol{y}$ of Σ^* a **string**. We denote the empty string by ε, and we assume that $\varepsilon \notin \Sigma$. A **discrete semimeasure** over Σ^* is a function $\mu\colon \Sigma^* \to [0, 1]$ such that $\sum_{\boldsymbol{y}\in\Sigma^*} \mu(\boldsymbol{y}) \leq 1$ (Bauwens, 2013; Icard, 2020). If the semimeasure of all strings sums to one, i.e., $\sum_{\boldsymbol{y}\in\Sigma^*} \mu(\boldsymbol{y}) = 1$, then μ is called a **probability measure**.[3] Finally, for any alphabet Σ, we define the set $\Sigma_\varepsilon \stackrel{\text{def}}{=} \Sigma \cup \{\varepsilon\}$.

Definition 2.1. *A **language model** (LM) p is a semimeasure over Σ^*. If p is a probability measure, it is called a **tight** language model.*

Definition 2.2. *Two LMs p and q are **weakly equivalent** if $p(\boldsymbol{y}) = q(\boldsymbol{y})$ for all $\boldsymbol{y} \in \Sigma^*$.*

Most modern LMs are autoregressive, i.e., they define $p(\boldsymbol{y})$ through conditional distributions of the next symbol given the string produced so far and the probability of ending the string, i.e.,

$$p(\boldsymbol{y}) \stackrel{\text{def}}{=} p(\textsc{eos} \mid \boldsymbol{y}) \prod_{t=1}^{|\boldsymbol{y}|} p(y_t \mid \boldsymbol{y}_{<t}). \tag{1}$$

Here, EOS denotes the special end-of-string symbol, which specifies that the generation of a string has halted. The inclusion of EOS allows (but does not guarantee) that a language model p, autoregressively, constitutes a probability measure over Σ^* (Du et al., 2023); a model defined as in Eq. (1) may sum to *less than* 1 in a pathological case. For any alphabet Σ, we define the set $\overline{\Sigma} \stackrel{\text{def}}{=} \Sigma \cup \{\textsc{eos}\}$. The conditional probability distributions are usually defined based on *vectorial* representations of $\boldsymbol{y}_{<t}$ computed by a **language encoder** $\mathsf{enc}\colon \Sigma^* \to \mathbb{R}^D$ (Chan et al., 2024).

Definition 2.3. *A **representation-based** LM is any LM that can be written as an autoregressive language model (Eq. (1)) where the conditional distributions over the next symbol $\overline{y}_t \in \overline{\Sigma}$ are given by*

$$p(\overline{y}_t \mid \boldsymbol{y}_{<t}) \stackrel{\text{def}}{=} \mathbf{f}(\mathbf{E}\, \mathsf{enc}(\boldsymbol{y}_{<t}))_{\overline{y}_t}, \tag{2}$$

where $\mathsf{enc}\colon \Sigma^* \to \mathbb{R}^D$ is a language encoder, $\mathbf{E} \in \mathbb{R}^{|\overline{\Sigma}|\times D}$ is an output matrix, and $\mathbf{f}$ is a projection function.[4]

[3] This definition differs from Li and Vitányi's (2008) who instead define semimeasures over *prefix strings*.

[4] A common choice for $\mathbf{f}$ is the softmax. Since our analyses use rational arithmetic, we instead opt for sparsemax (Martins and Astudillo, 2016). However, all of our results can be extended to the use of the more common softmax function through the use of log activations and the extended real numbers $\mathbb{R} \cup \{-\infty, \infty\}$ (Svete et al., 2024).

2.2 Regular Language Models

Probabilistic finite-state automata are a well-understood probabilistic computational model.

Definition 2.4. *A **probabilistic finite-state automaton** (PFSA) is a tuple $(\Sigma, Q, \lambda, \rho, \delta)$ where Σ is an alphabet, Q is a finite set of states, $\delta \subseteq Q \times \Sigma \times \mathbb{Q}_{\geq 0} \times Q$ is a finite set of weighted transitions where we write transitions $(q, y, w, q') \in \delta$ as $q \xrightarrow{y/w} q'$, and $\lambda, \rho \colon Q \to \mathbb{Q}_{\geq 0}$ are functions that assign each state its initial and final weight, respectively. Moreover, for all states $q \in Q$, δ, λ and ρ satisfy $\sum_{q \in Q} \lambda(q) = 1$, and $\sum_{q \xrightarrow{y/w} q' \in \delta} w + \rho(q) = 1$.*

Next, we will define some basic concepts related to PFSAs. A PFSA $\mathcal{A} = (\Sigma, Q, \lambda, \rho, \delta)$ is **deterministic** if $|\{q \mid \lambda(q) > 0\}| = 1$ and, for every $q \in Q, y \in \Sigma$, there is at most one $q' \in Q$ such that $q \xrightarrow{y/w} q' \in \delta$ with $w > 0$.[5] Any state q where $\lambda(q) > 0$ is called an **initial state**, and if $\rho(q) > 0$, it is called a **final state**. A **path** $\boldsymbol{\pi}$ of length N is a sequence of subsequent transitions in $\mathcal{A}$, denoted as

$$q_1 \xrightarrow{y_1/w_1} q_2 \xrightarrow{y_2/w_2} q_3 \cdots q_N \xrightarrow{y_N/w_N} q_{N+1}. \quad (3)$$

The **yield** of a path is $\mathbf{s}(\boldsymbol{\pi}) \stackrel{\text{def}}{=} y_1 \cdots y_N$. The **prefix weight** $\widetilde{\mathrm{w}}$ of a path $\boldsymbol{\pi}$ is the product of the transition and initial weights, whereas the **weight** of a path additionally has the final weight multiplied in. In symbols, this means

$$\widetilde{\mathrm{w}}(\boldsymbol{\pi}) \stackrel{\text{def}}{=} \prod_{n=0}^{N} w_n, \quad (4) \qquad \mathrm{w}(\boldsymbol{\pi}) \stackrel{\text{def}}{=} \prod_{n=0}^{N+1} w_n, \quad (5)$$

with $w_0 \stackrel{\text{def}}{=} \lambda(q_1)$ and $w_{N+1} \stackrel{\text{def}}{=} \rho(q_{N+1})$. We write $\Pi(\mathcal{A})$ for the set of all paths in $\mathcal{A}$ and we write $\Pi(\mathcal{A}, \boldsymbol{y})$ for the set of all paths in $\mathcal{A}$ with yield $\boldsymbol{y}$. The sum of weights of all paths that yield a certain string $\boldsymbol{y} \in \Sigma^*$ is called the **stringsum**, which we write as

$$\mathcal{A}(\boldsymbol{y}) \stackrel{\text{def}}{=} \sum_{\boldsymbol{\pi} \in \Pi(\mathcal{A}, \boldsymbol{y})} \mathrm{w}(\boldsymbol{\pi}). \quad (6)$$

The stringsum gives the probability of the string $\boldsymbol{y}$. This way, PFSAs induce a particularly well-understood family of LMs.

[5] In this paper, we do *not* distinguish between a transition for a given symbol with weight $w = 0$ and the absence of a transition for that symbol. That is, we assume there always exists a transition $q \xrightarrow{y/w} q' \in \delta$ for any $q, q' \in Q$ and $y \in \Sigma$, albeit possibly with $w = 0$. Such a choice turns out to be useful in our technical exposition.

Definition 2.5. *An LM p is a **regular** LM if there exists a PFSA $\mathcal{A}$ whose induced language model $p_{\mathcal{A}}$ is weakly equivalent to p.*

PFSAs and non-determinism. Although PFSAs share many properties with unweighted (boolean-weighted) finite-state automata, one important difference relates to determinization. In the unweighted case, the class of deterministic and non-deterministic FSAs are equivalent, i.e., any non-deterministic FSA has an equivalent deterministic FSA that accepts the same language. This result, however, does not hold for PFSAs: There exist PFSAs that admit no deterministic equivalent (Mohri, 1997; Buchsbaum et al., 2000), meaning that non-deterministic PFSAs are strictly more expressive than deterministic ones.

Computing string probabilities under non-determinism. Autoregressive LMs (cf. Eq. (1)) and PFSAs fall under the larger framework of models that specify probability distributions *implicitly* (Icard, 2020).[6] However, in autoregressive neural LMs with fixed precision, only one sequence of computational actions can yield a particular string, meaning that they can only model deterministic weighted regular languages (Svete and Cotterell, 2023). On the other hand, non-deterministic PFSAs compute string probabilities by *additively* combining the probabilities of all paths that yield the same string, and hence lie outside the grasp of such neural models.[7] As we show later, CoT reasoning provides a principled way to overcome this limitation and allow fixed-precision neural models to simulate non-deterministic automata.

2.3 Regular Functions

We now define a finite-state machine that, in addition to scanning, also *outputs* strings.

Definition 2.6. *A **finite-state transducer** (FST) is a 6-tuple $\mathcal{T} = (Q, \Sigma, \Xi, I, F, \delta)$, where Q is a finite set of **states**, Σ is an alphabet of **input symbols**, Δ is an alphabet of **output symbols**, $I, F \subseteq Q$ are sets of initial and final states, respectively, and $\delta \subseteq Q \times \Sigma_\varepsilon \times \Delta_\varepsilon \times Q$ is a set of transitions.*

Similar to PFSA transitions, we write FST transitions of the form $(q, x, y, q') \in \delta$ as $q \xrightarrow{x:y} q'$.

[6] Other examples of such models include hidden Markov models, which are equivalent to PFSAs (Icard, 2020).

[7] Note that RNN LMs with linearly bounded precision can simulate non-deterministic PFSAs in real-time by encoding a probability distribution over the PFSA's current state in the hidden state of the RNN (Svete et al., 2024).